

شرح مشروع HifzTracker

أهلاً بك! هذا الملف يحتوي على شرح مفصل ومبسط لمشروع **HifzTracker**، وكيف تعمل الملفات مع بعضها البعض لبناء النظام بالكامل.

1. نظرة عامة على المشروع (Overview)

المشروع عبارة عن منصة لتسميع ومراجعة القرآن الكريم (Hifz Tracker). مبني باستخدام إطار عمل **Django** (بايثون). الهدف الأساسي منه هو الربط بين الطلاب والمعلمين في حلقات تحفيظ، حيث يقوم المعلم بتعيين مهام (تسميع أو مراجعة)، ويقوم الطالب بتسجيل صوته ورفعها، ثم يقوم المعلم بالتصحيح.

2. الهيكل العام للمجلدات (Folder Structure)

المشروع مقسم لمجلدات رئيسية، كل مجلد له وظيفة محددة:

hifztracker/ (مجلد الإعدادات الرئيسي)

هذا هو "عقل" المشروع، ويحتوي على الإعدادات العامة.

- settings.py**: ملف الإعدادات. فيه نحدد قاعدة البيانات، التطبيقات المثبتة (Installed Apps)، إعدادات البريد الإلكتروني، واللغة.
- urls.py**: "وجه المرور" الرئيسي. أي رابط يطلبه المستخدم (مثل **/login** أو **/dashboard**) يمر من هنا أولاً ليتم توجيهه للتطبيق المناسب.
- asgi.py / wsgi.py**: ملفات تستخدم عند رفع الموقع على السيرفر (Production) ليعمل مع خوادم الويب.

apps/ (التطبيقات)

هنا يوجد "اللحم والعظم" للمشروع. تم تقسيم الكود لتطبيقات صغيرة لتنظيمه. يحتوي الآن على التطبيق الأساسي:

- accounts**: يدير المستخدمين، الحلقات، المهام، والتسليمات. هو المحرك الأساسي للمنصة.

يحتوي على ملفات **HTML**. هي الصفحات التي يراها المستخدم (واجهة الموقع).

- accounts/ : صفحات الدخول والتسجيل.
- students/ : لوحة تحكم الطالب.
- teachers/ : لوحة تحكم المعلم.

- static : ملفات التصميم (CSS)، الجافاسكريبت (JS)، والصور الثابتة (الوجو مثلاً).
- media : الملفات التي يرفعها المستخدمون (مثل صور البروفايل، أو تسجيلات التسميع الصوتية).

3. شرح التفاصيل والملفات (File by File)

سنركز على تطبيق **apps/accounts** لأنه المحرك الوحيد للموقع حالياً.

apps/accounts/models.py (قاعدة البيانات) 📄

هذا الملف يصف "شكل البيانات" في النظام. كل **class** هنا يتحول لجدول في قاعدة البيانات. أهم الجداول (Models):

- Profile** : يربط كل مستخدم (User) ببيانات إضافية مثل: هل هو **طالب** أم **معلم**؟ ما هي حلقاته؟ صورته؟ هو العمود الفقري لتحديد الصلاحيات.
- Halaqa (الحلقة)** : تمثل فصل التحفيظ. لها اسم، وترتبط الطلاب بالمعلمين.
- Review & Recitation** : هذه هي "الواجبات". المعلم ينشئ Recitation (مثلاً: سورة البقرة من آية 1 لـ 10).
- Submission** : عندما يقوم الطالب بتسجيل صوته ورفعه، يتم حفظه هنا.
- Attendance** : لتسجيل حضور وغياب الطلاب.

apps/accounts/views.py (المنطق والتحكم) 📄

هذا الملف هو "الدينامو". يحتوي على الدوال (Functions) التي تنفذ الأوامر. أهم الدوال:

1. `register_view & login_view` : تدير عملية الدخول وإنشاء الحساب. ملاحظة: تحتوي على منطق إرسال كود تحقق (OTP) للإيميل.
2. `student_dashboard` : تجمع بيانات الطالب: مهامه المعلقة، درجاته، نسبة حضوره.
3. `teacher_dashboard` : تجمع بيانات المعلم: حلقاته، التسليمات الجديدة.
4. `submit_task` : تستقبل ملف الصوت من الطالب وتحفظه في قاعدة البيانات.

4. كيف تتصل الملفات ببعضها؟ (The Flow)

لنتخيل سيناريو "طالب يدخل ليسلم واجب":

1. المتصفح (Browser): الطالب يطلب رابط الموقع `/dashboard/`.
2. `hifztracker/urls.py` : يستقبل الطلب، يوجهه إلى `apps.accounts.urls`.
3. `apps/accounts/urls.py` : يوجهه للدالة `student_dashboard` في ملف `views.py`.
4. `views.py` : تطلب البيانات من `Models`، وترسلها لملف `HTML`.
5. `student_dashboard.html` : يعرض الصفحة للطالب.

ملخص بسيط

- **Models**: شكل البيانات (جداول).
 - **Views**: العقل المدبر (كود بايثون).
 - **Templates**: الشكل الخارجي (HTML).
 - **Urls**: العناوين والروابط.
- هذا هو باختصار هيكل مشروعك! هو منظم جداً ويعتمد على مبدأ فصل المسؤوليات (كل ملف له وظيفة محددة).